

DOKUMEN TEKNIS & PANDUAN INFRASTRUKTUR

Sistem Telemetry & Executive IoT Dashboard Jamkrida Energy

Judul Dokumen	Panduan Penyusunan Infrastruktur, Stack Teknologi, API Endpoint, & Desain Database
Versi Aplikasi	1.1 (Production-Ready)
Tanggal Rilis	23 Juni 2026
Repositori GitHub	https://github.com/suppfaiz/iotdashboard-jjt.git
Target Server	Linux Ubuntu VPS / Docker Container Engine
Penyusun	Antigravity AI Assistant

DAFTAR ISI

Topik Bahasan	Halaman
1. Stack Teknologi Aplikasi	3
2. Fitur Utama Sistem Dashboard	3
3. Dokumentasi API & Endpoint Pengendalian	4
4. Pemetaan URL & Endpoint Aplikasi	4
5. Panduan Instalasi VPS Terperinci (Step-by-Step)	5
6. Desain Database & Relasi Tabel Terperinci	6
7. Pengaturan Kredensial Keamanan (.env)	8
8. Command Operasional Sistem (Cheatsheet)	9
8.1. Mengelola Layanan Docker (VPS)	9
8.2. Prosedur Update Aplikasi di VPS	9
9. Lampiran: Screenshot Halaman Aplikasi	10
9.1. Halaman Dashboard Utama (Overview & Analytics)	10
9.2. Halaman Daftar Alat (Devices List)	11
9.3. Halaman Audit Logs (Daily & Hourly Logs)	12
9.4. Halaman Export Laporan (Reports Page)	13
9.5. Halaman Pengaturan (System Settings)	14
10. Panduan Pengujian & Troubleshooting (Debug MQTT & WebSockets)	15

1. STACK TEKNOLOGI APLIKASI

Aplikasi IoT Dashboard Jamkrida Energy dibangun menggunakan kombinasi backend tangguh, protokol pertukaran data ringan berstandar industri, serta client mikrokontroler hardware dengan performa real-time:

- **Back-end Web Framework:** Laravel 13 berjalan pada runtime interpreter PHP 8.4+. Framework ini menangani manajemen perangkat keras, pembuatan kode sketch C++ Arduino ESP32 secara dinamis, pengelolaan database SQLite, pemrosesan antrean *background worker* (Supervisor), dan visualisasi data front-end.
- **Real-time WebSocket Server:** Laravel Reverb. Layanan WebSocket server terintegrasi dengan latensi sangat rendah untuk menyiarkan telemetry dari *background worker* langsung ke browser client tanpa polling konstan. Berjalan di port 8085.
- **MQTT Message Broker (Telemetry Queue):** Eclipse Mosquitto (v2.x). Berperan sebagai broker pengumpul data telemetry sensor dari modul mikrokontroler. Broker ini dioperasikan dengan mematikan akses anonim untuk menjamin keamanan pengiriman perintah kontrol dan telemetry.
- **Database Engine:** SQLite 3. Dipilih karena menawarkan efisiensi tinggi, berkecepatan tinggi, minim penggunaan memori server, serta bebas dari beban overhead manajemen server DBMS eksternal yang besar.
- **Web-based Database Manager:** Adminer. Tool pengelola database ringan berbasis web. Berjalan di port 8082 dan diamankan dengan form login wajib menggunakan password environment ADMINER_PASSWORD.
- **Front-end Stack:** TailwindCSS (compilation bundler via Vite), Vanilla JS untuk logika WebSockets subscriber, dan Chart.js untuk merender visualisasi grafik konsumsi kWh.
- **Hardware Telemetry Client:** ESP32 NodeMCU terhubung dengan modul meter daya PZEM-004T v3.0 yang membaca parameter Voltase, Arus, Daya Aktif (Watt), dan Akumulasi Energi (kWh) melalui port komunikasi UART Serial.

2. FITUR UTAMA SISTEM DASHBOARD

Sistem ini dirancang untuk monitoring berkelanjutan dan kontrol periferal hardware dari jarak jauh:

- **Real-time Telemetry Monitor:** Pembacaan sensor daya listrik secara instan (Voltase, Arus, Watt, kWh) yang dikirimkan oleh mikrokontroler lapangan dan disebarkan ke antarmuka pengguna tanpa memicu reload halaman web.
- **Kalkulasi Estimasi Biaya Tagihan:** Penghitungan biaya konsumsi listrik harian secara dinamis berdasarkan total akumulasi pemakaian energi kWh dikalikan tarif listrik per kWh (misalnya tarif PLN Golongan R-1). Tarif dasar PLN dapat diubah secara dinamis dari menu pengaturan.
- **Top 3 Perangkat Konsumsi Tertinggi:** Peringkat widget real-time yang secara otomatis menyaring dan menampilkan 3 perangkat sensor dengan konsumsi daya Watt/kWh terbesar.
- **Status Konektivitas Deteksi Otomatis (Keep-Alive):** Indikator status perangkat (Aktif/Nonaktif) yang diperbarui otomatis setiap kali mendeteksi pesan telemetry. Apabila perangkat tidak mengirimkan data telemetry dalam kurun waktu 15 detik, status di dashboard akan berganti menjadi *Inactive*.

- **Grafik Tren Konsumsi Interaktif:** Visualisasi grafik Chart.js dengan fitur filter waktu (Harian, Bulanan, Tahunan) serta tombol konversi unit (menampilkan nilai data konsumsi dalam kWh atau akumulasi Rupiah).
- **Dynamic firmware Generator:** Menu konfigurasi firmware di mana administrator cukup memasukkan nama SSID & password WiFi, lalu menekan tombol generator untuk menghasilkan file C++ Arduino IDE lengkap yang sudah terintegrasi dengan kredensial WiFi dan username/password koneksi MQTT ke broker secara dinamis.
- **Perintah Jarak Jauh (Remote Command MQTT):** Tombol interaktif untuk memicu perintah ****Restart ESP32**** dan ****Reset Akumulasi kWh**** (nol kembali) dari dashboard web ke perangkat keras fisik di lapangan.

3. DOKUMENTASI API & ENDPOINT PENGENDALIAN

Semua rute API kontrol dilindungi oleh middleware otentikasi (auth) dan hak akses administrator (admin). Berikut adalah daftar endpoint yang digunakan sistem untuk berkomunikasi dengan frontend dan melakukan kontrol perangkat:

Endpoint Rute	Method	Fungsi / Kegunaan	Keterangan Aksi & Payload
/ atau /dashboard	GET	Halaman Dashboard Utama	Menampilkan grafik Chart.js, widget real-time, dan Top 3.
/devices	GET	Daftar Alat	Menampilkan seluruh daftar sensor yang terdaftar.
/devices/{device}/restart	POST	Restart Perangkat Jauh	Menerbitkan perintah MQTT {"cmd": "restart"} ke topic cmd/{device_id} untuk pemicuan restart ESP32.
/devices/{device}/reset-energy	POST	Reset Nilai kWh	Menerbitkan perintah MQTT {"cmd": "reset_energy"} ke topic cmd/{device_id} dan menghapus cache lokal Laravel.
/devices/{device}/trigger-ota	POST	Trigger Update Firmware	Menerbitkan perintah {"cmd": "update_firmware", "url": ""} agar ESP32 mengunduh berkas firmware baru.
/logs	GET	Audit Logs	Menerima parameter query ?type=hourly atau ?type=daily untuk mengambil data log historis.

4. PEMETAAN URL & ENDPOINT APLIKASI

Setelah melakukan instalasi di VPS menggunakan script `install.sh`, sistem akan memetakan port container ke port host secara otomatis dengan rincian sebagai berikut:

Layanan	Protokol & Port	URL Akses / Endpoint
Dashboard Web App	HTTP (Port 80)	http://<VPS_IP>/
Adminer DB Manager	HTTP (Port 8082)	http://<VPS_IP>:8082/

Layanan	Protokol & Port	URL Akses / Endpoint
Reverb WebSockets	WS/WSS (Port 8085)	ws://<VPS_IP>:8085/app/<KEY>
Mosquitto Broker	TCP Socket (Port 1883)	mqtt://<VPS_IP>:1883/

5. PANDUAN INSTALASI VPS TERPERINCI (STEP-BY-STEP)

Persyaratan Awal Server: Sistem membutuhkan server Linux Ubuntu VPS dengan minimal 1 CPU Core, RAM 1GB, dan port 80, 8082, 8085, serta 1883 dalam kondisi tidak terblokir oleh Firewall (*Security Group* atau `ufw`).

Langkah 5.1: Hubungkan ke VPS via SSH

Akses terminal Anda dan lakukan koneksi jarak jauh ke VPS menggunakan kunci SSH atau password root Anda:

```
ssh username@vps_ip_address
```

Langkah 5.2: Pasang Git dan Unduh Repository Kode

Perbarui paket VPS dan lakukan clone repository kode sumber dashboard ke direktori kerja server menggunakan URL repositori GitHub resmi:

```
sudo apt update && sudo apt install -y git
cd /var/www
git clone https://github.com/suppfaiz/iotdashboard-jjt.git dashboard-iot
cd dashboard-iot
```

Langkah 5.3: Berikan Izin Eksekusi & Jalankan Installer

Installer otomatis akan mengonfigurasi segalanya termasuk pembuatan docker image, pembuatan file database SQLite, konfigurasi Reverb, pembuatan akun broker MQTT, serta pengacakan password otomatis:

```
chmod +x install.sh
./install.sh
```

Masukkan IP Publik VPS atau Domain Anda saat diminta oleh installer untuk konfigurasi koneksi browser ke WebSockets.

Langkah 5.4: Konfigurasi Laravel Scheduler (Cron Job)

Guna memicu kalkulasi rekap log per jam dan harian secara berkala di production, tambahkan baris cron job di server VPS Anda:

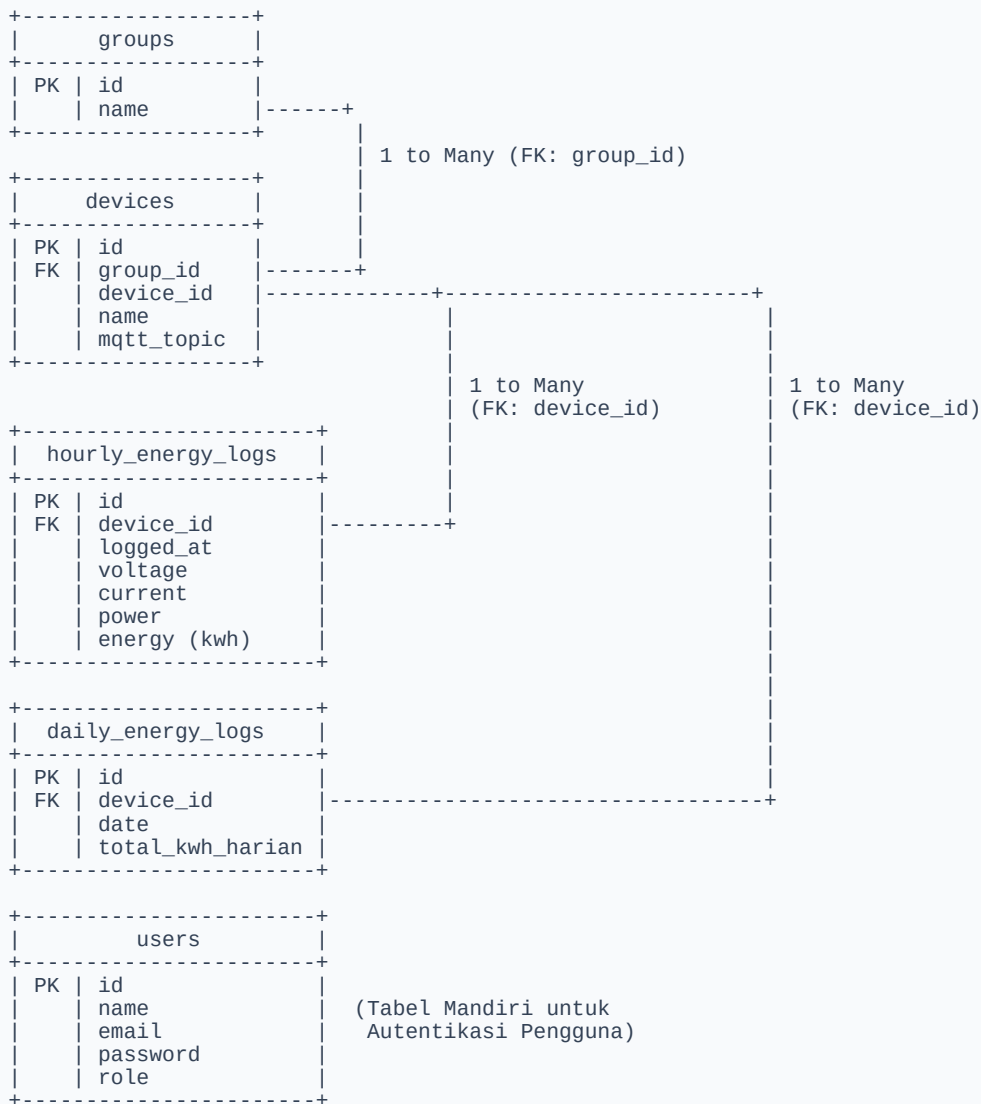
```
# Buka konfigurasi crontab server
crontab -e

# Tambahkan baris berikut di baris paling bawah, lalu simpan:
* * * * * cd /var/www/dashboard-iot && docker compose exec -T app php artisan schedule:run >> /dev/null 2>&1
```


6. DESAIN DATABASE & RELASI TABEL TERPERINCI

Database menggunakan engine SQLite 3 (pada file volume database/database.sqlite). Relasi antar tabel dirancang secara teratur dengan relasi kunci asing (*Foreign Keys*) untuk menjamin integritas data referensial.

Diagram Relasi Tabel (Entity Relationship Diagram - ERD)



Deskripsi Kolom & Kunci Relasi

1. Tabel `users` (Manajemen Akun Pengguna Web)

Menampung data kredensial user untuk kebutuhan autentikasi masuk ke dashboard web.

- **id** (INTEGER, Primary Key, Auto Increment): ID unik pengguna.

- name (VARCHAR): Nama lengkap pengguna web.
- email (VARCHAR, Unique): Email login pengguna (Default: admin@admin.com).
- password (VARCHAR): Hash password bcrypt pengguna (Default: password).
- role (VARCHAR): Peran hak akses (misal: admin).

2. Tabel `groups` (Wilayah/Divisi Kerja)

Menampung data kategori area operasional perangkat sensor di lingkungan kantor.

- `id` (INTEGER, Primary Key, Auto Increment): ID unik divisi kerja.
- `name` (VARCHAR): Nama divisi (misal: OPERASIONAL, FINANCE, HR&GA).

3. Tabel `devices` (Daftar Perangkat Alat Sensor)

Menyimpan detail konfigurasi setiap perangkat sensor IoT.

- `id` (INTEGER, Primary Key): ID unik autoincrement.
- `device_id` (VARCHAR, Unique): String ID unik perangkat keras (misal: dev_wFXTw0B0).
- `name` (VARCHAR): Nama identitas perangkat.
- `group_id` (INTEGER, Foreign Key): Relasi kunci asing merujuk ke `groups.id`. Menggunakan aturan cascade delete.
- `mqtt_topic` (VARCHAR): Topic khusus bagi perangkat untuk mempublikasikan datanya.
- `provisioning_code` (TEXT): Berkas salinan kode C++ Arduino ESP32.

4. Tabel `hourly\_energy\_logs` (Logs Snapshot per 1 Jam)

Menyimpan data parameter sensor listrik yang direkam secara periodik di awal setiap jam.

- `id` (INTEGER, Primary Key): ID log unik.
- `device_id` (INTEGER, Foreign Key): Relasi kunci asing merujuk ke `devices.id` dengan aturan cascade delete.
- `logged_at` (TIMESTAMP): Waktu pencatatan dengan pembulatan awal jam (format: YYYY-MM-DD HH:00:00).
- `voltage` (DECIMAL 8,2): Tegangan terukur (V).
- `current` (DECIMAL 8,4): Arus terukur (A).
- `power` (DECIMAL 8,2): Daya aktif terukur (Watt).
- `energy` (DECIMAL 10,4): Akumulasi pemakaian energi kWh pada jam pencatatan.

5. Tabel `daily\_energy\_logs` (Rekapitulasi Konsumsi Harian)

Menyimpan rekap total pemakaian energi bersih harian per perangkat untuk kebutuhan statistik chart.

- `id` (INTEGER, Primary Key): ID rekap unik.
- `device_id` (INTEGER, Foreign Key): Relasi kunci asing merujuk ke `devices.id` dengan aturan cascade delete.
- `date` (DATE): Tanggal pencatatan rekapitulasi harian (format: YYYY-MM-DD).
- `total_kwh_harian` (DECIMAL 10,4): Total konsumsi energi bersih harian (kWh) untuk perangkat terkait.

7. PENGATURAN KREDENSIAL KEAMANAN (.ENV)

Semua kredensial bersifat rahasia dan disimpan di dalam file `.env` di root folder setelah instalasi selesai:

Layanan	Key Variabel .env	Keterangan & Cara Akses
Web Admin Dashboard	- (Default Akun Laravel)	Email default: admin@admin.com Password default: password <i>Sangat disarankan segera mengganti password di menu profil setelah login pertama.</i>
Database SQLite	DB_PASSWORD	Digunakan untuk mengenkripsi koneksi database internal. Di-generate acak oleh installer.
Database GUI (Adminer)	ADMINER_PASSWORD	Password wajib saat login ke panel Adminer pada Port 8082 . Mencegah akses database tanpa izin dari luar.
MQTT User	MQTT_USERNAME	User default: jamkrida_sensor (untuk komunikasi ESP32 dan Laravel worker).
MQTT Password	MQTT_PASSWORD	Password acak broker MQTT yang di-generate otomatis oleh installer untuk proteksi broker Mosquitto.

8. COMMAND OPERASIONAL SISTEM (CHEATSHEET)

Jalankan perintah berikut pada folder root VPS project untuk administrasi sistem:

8.1. Mengelola Layanan Docker (VPS)

Tujuan	Command Terminal
Menyalakan seluruh sistem di background	<code>docker compose up -d</code>
Mematikan seluruh layanan sistem	<code>docker compose down</code>
Melihat log aktivitas kontainer secara real-time	<code>docker compose logs -f</code>
Membangun ulang kontainer setelah perubahan kode	<code>docker compose up -d --build</code>
Memeriksa status background worker (Supervisor)	<code>docker compose exec app supervisorctl status</code>

8.2. Prosedur Update Aplikasi di VPS

Jika Anda melakukan perubahan kode di repositori lokal dan telah melakukan push ke GitHub, jalankan urutan perintah berikut di terminal VPS untuk menerapkan pembaruan sistem secara menyeluruh:

```
# 1. Masuk ke folder proyek dan tarik kode terbaru dari GitHub
cd /var/www/dashboard-iot
git pull origin main

# 2. Matikan container dan bangun ulang (rebuild) image Docker
docker compose down
docker compose up -d --build

# 3. Jalankan migrasi database SQLite di dalam container
docker compose exec app php artisan migrate --force

# 4. Bersihkan cache konfigurasi, route, dan view Laravel
docker compose exec app php artisan config:clear
docker compose exec app php artisan route:clear
docker compose exec app php artisan view:clear

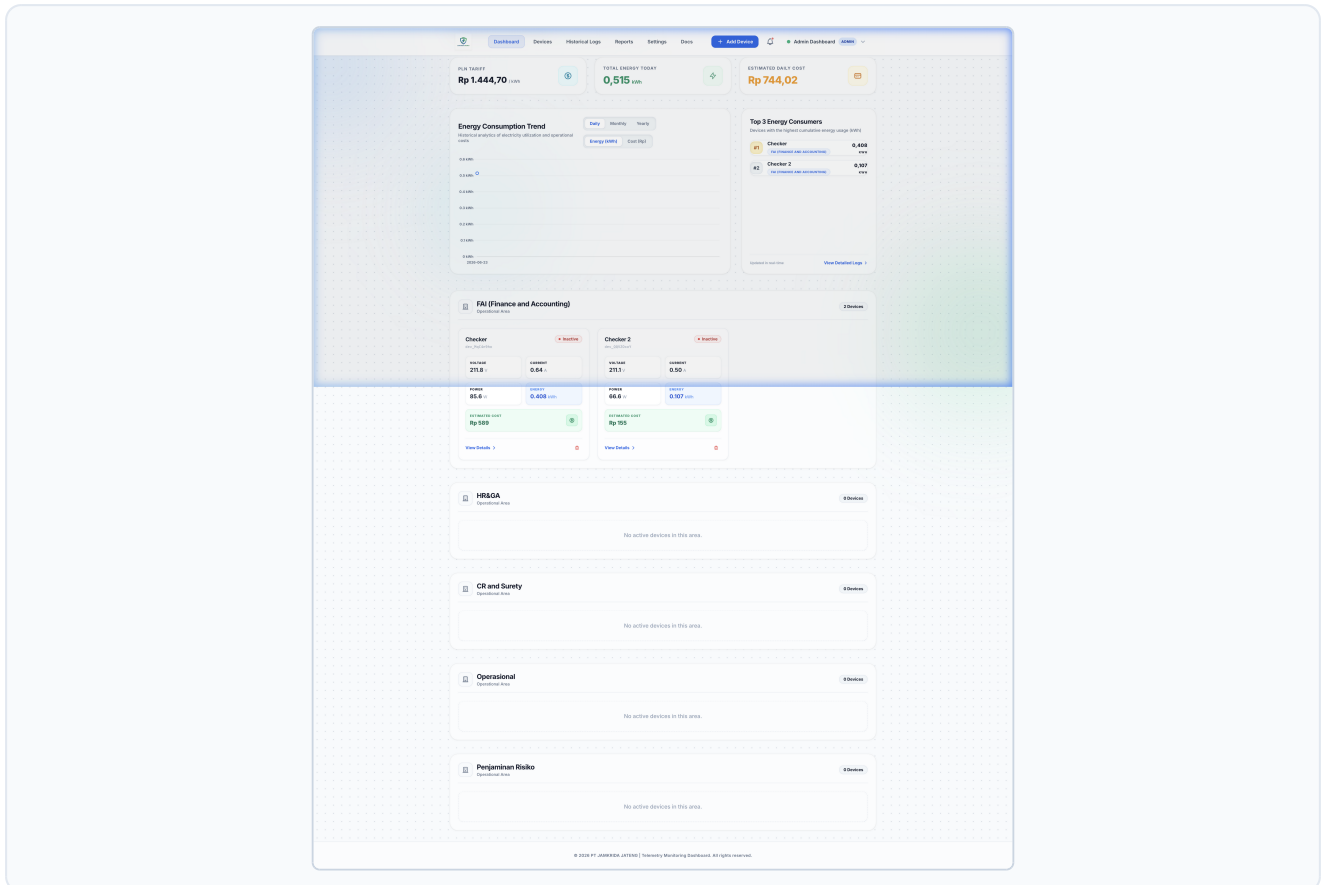
# 5. Periksa status background worker (Supervisor)
docker compose exec app supervisorctl status
```

9. LAMPIRAN: SCREENSHOT HALAMAN APLIKASI

Berikut adalah tampilan antarmuka visual (halaman web) dari Executive IoT Dashboard Jamkrida Energy:

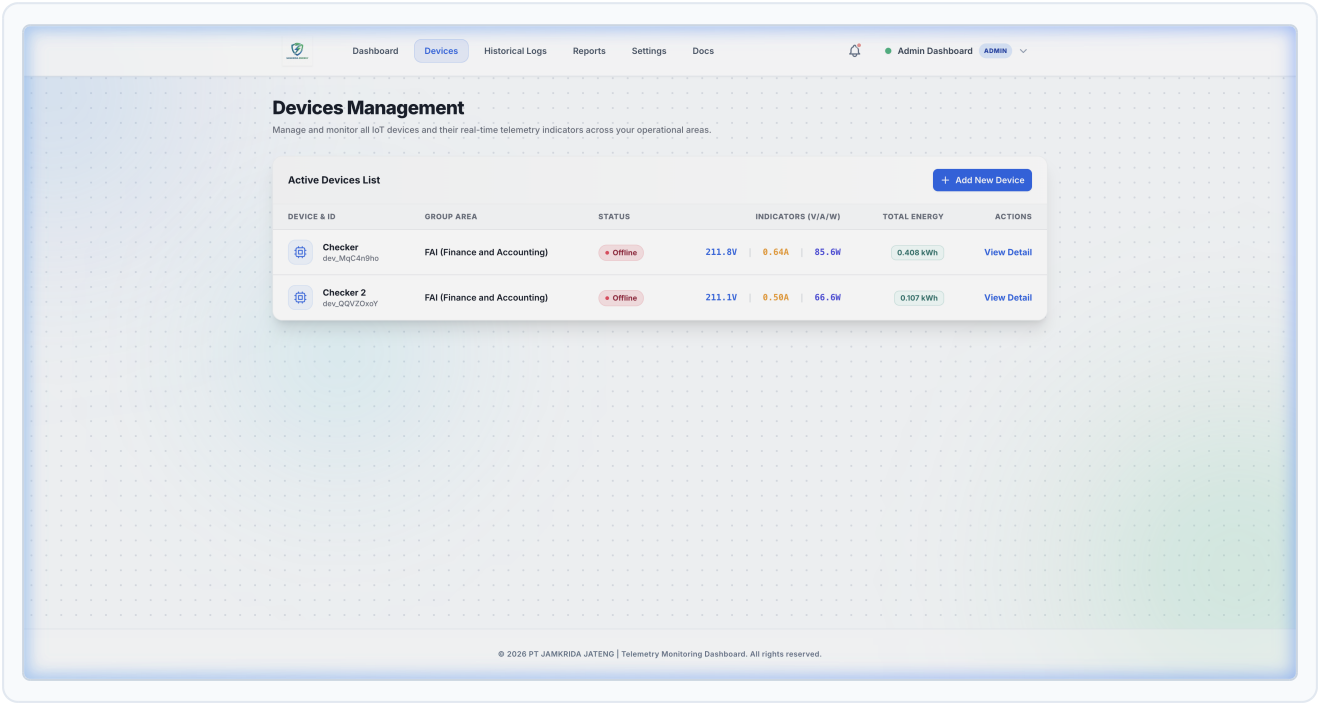
9.1. Halaman Dashboard Utama (Overview & Analytics)

Menyajikan widget real-time, grafik tren interaktif Chart.js, serta peringkat Top 3 penggunaan energi harian.



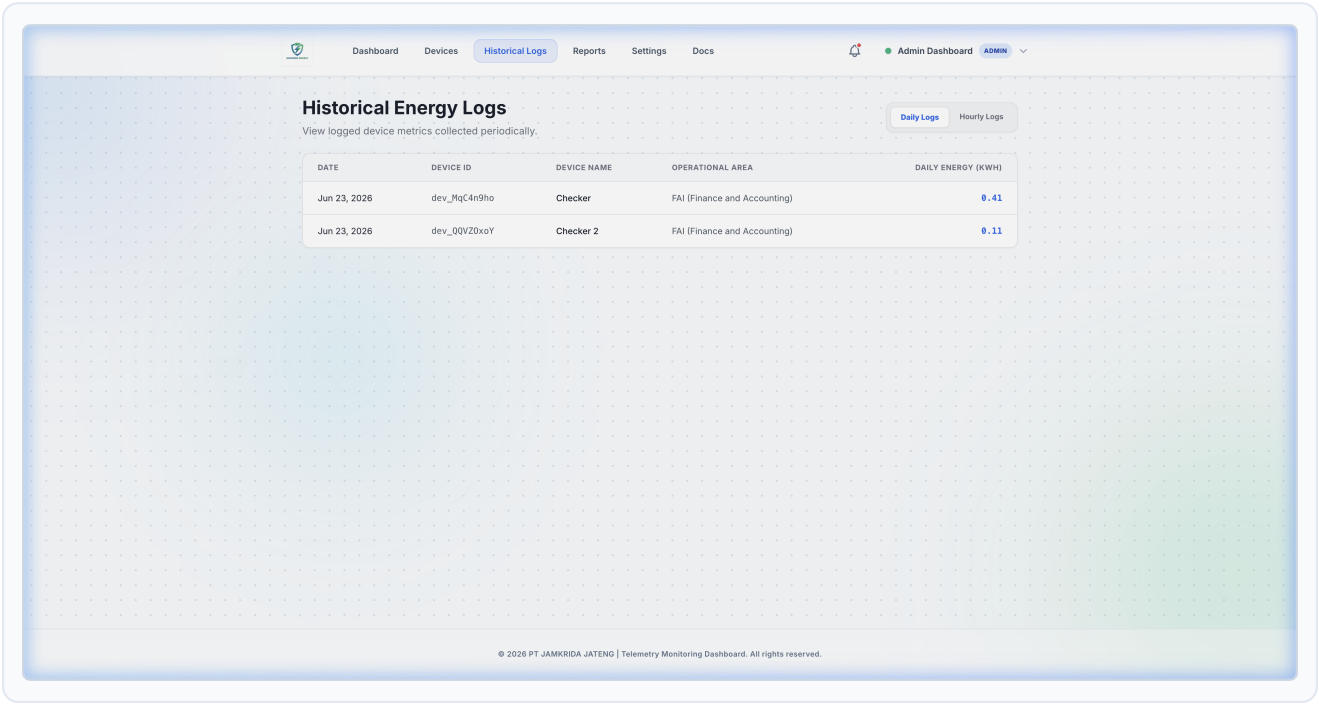
9.2. Halaman Daftar Alat (Devices List)

Menampilkan semua alat sensor yang aktif maupun nonaktif beserta topic MQTT-nya.



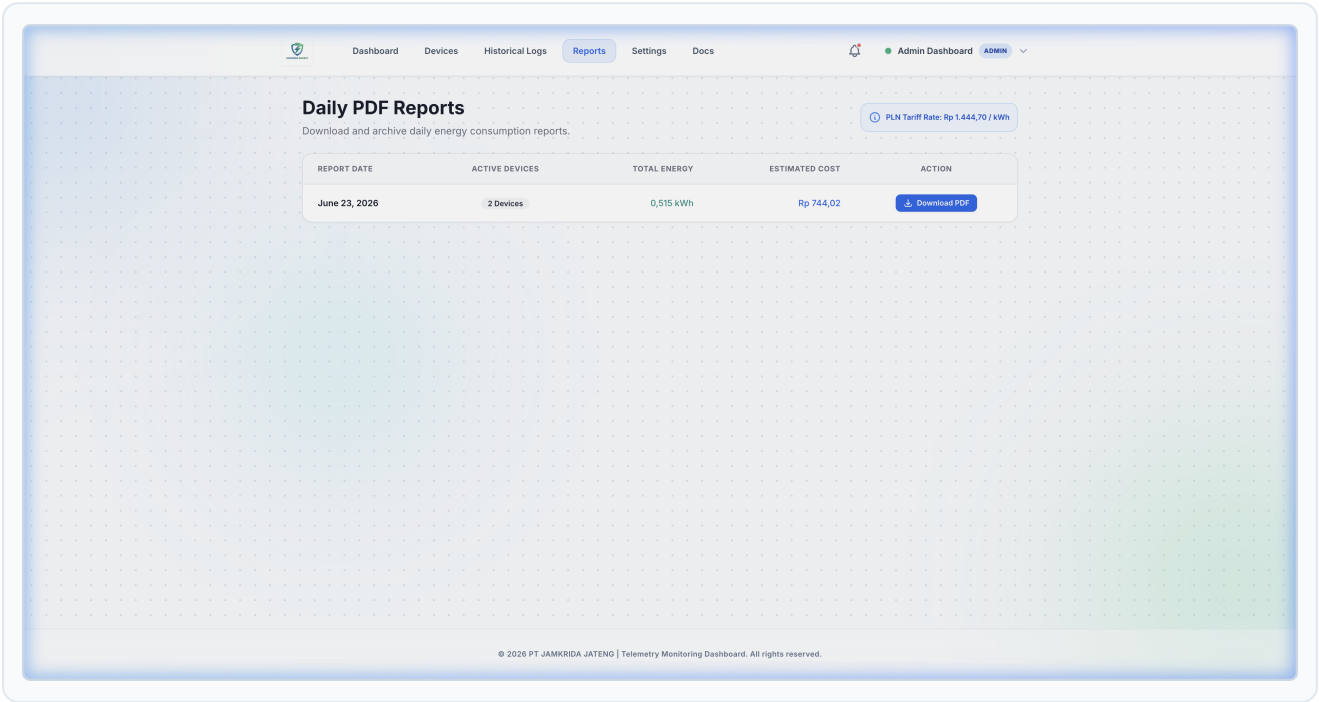
9.3. Halaman Audit Logs (Daily & Hourly Logs)

Menyajikan rekaman audit logs yang tersimpan di database per tanggal dan awal jam secara kronologis.



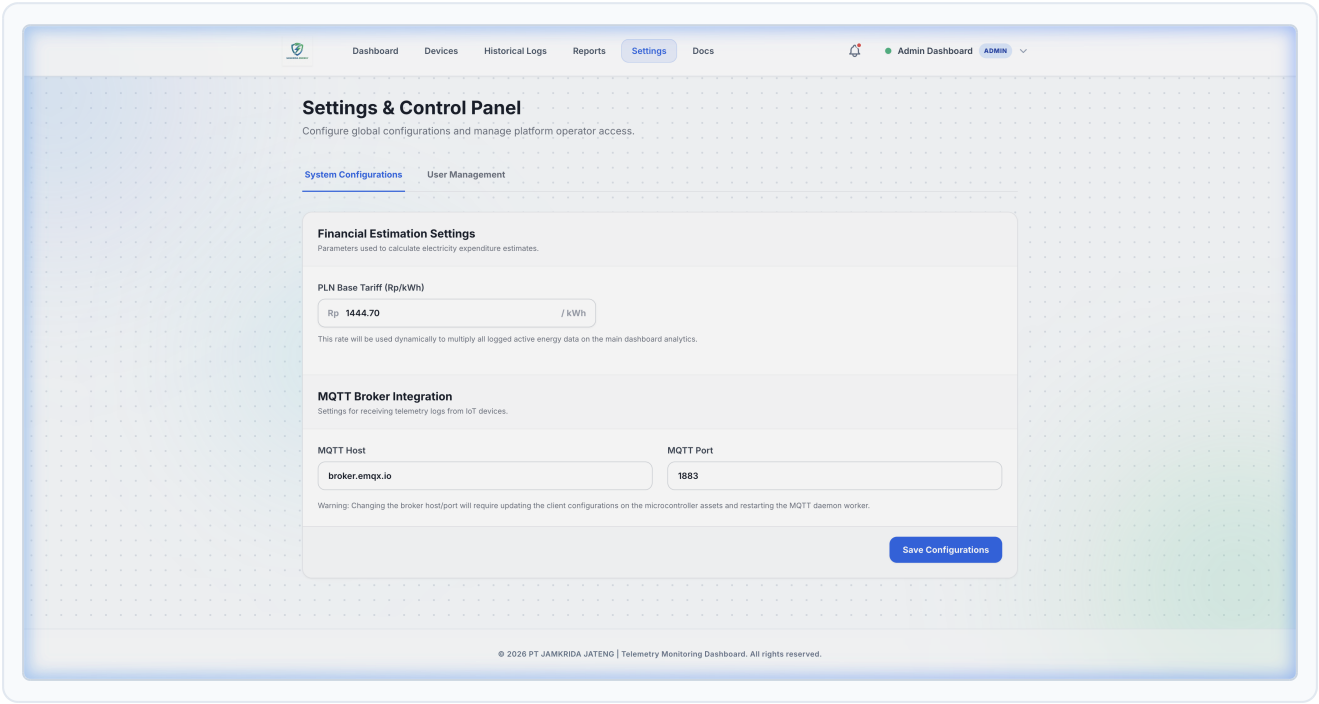
9.4. Halaman Export Laporan (Reports Page)

Menawarkan fitur untuk mengunduh rekap pemakaian energi harian dalam bentuk PDF/Excel.



9.5. Halaman Pengaturan (System Settings)

Konfigurasi tarif dasar listrik PLN, credential MQTT broker, serta manajemen akun user.



10. PANDUAN PENGUJIAN & TROUBLESHOOTING (DEBUG MQTT & WEBSOCKETS)

Guna memudahkan proses pemeliharaan sistem di lapangan, berikut adalah prosedur pengujian modul telemetry dan WebSocket:

10.1. Pengujian Koneksi Telemetry Broker (MQTT CLI)

Anda dapat mensimulasikan telemetry pengiriman data dari laptop/PC lokal untuk menguji apakah broker MQTT menerima dan memproses data dengan benar:

```
# Melakukan subscribe ke topic telemetry dari terminal eksternal (pastikan mosquitto-clients terpasang)
mosquitto_sub -h <VPS_IP> -p 1883 -u jamkrida_sensor -P <MQTT_PASSWORD> -t "telemetry/+"

# Mengirim data dummy telemetry (Voltase, Arus, Daya, kWh) ke topic broker
mosquitto_pub -h <VPS_IP> -p 1883 -u jamkrida_sensor -P <MQTT_PASSWORD> -t "telemetry/dev_test01" -m '{"voltage":220.5,"current":1.25,"power":275.62,"energy":0.015}'
```

10.2. Troubleshooting Layanan Worker & WebSockets

Apabila grafik telemetry real-time di browser tidak berganti nilainya secara berkala:

1. **Periksa Antrean Worker Laravel:** Pastikan background listener MQTT berjalan dengan mengecek status supervisor di container:

```
docker compose exec app supervisorctl status
```

Layanan mqtt-listener dan reverb harus berstatus **RUNNING**.

2. **Periksa Log Aplikasi Laravel:**

```
docker compose exec app tail -n 50 storage/logs/laravel.log
```

Periksa jika ada kendala koneksi database, port terblokir, atau error runtime PHP.

3. **Buka Console Browser (F12):** Periksa tab *Network -> WS* (WebSockets). Hubungkan ke server WebSocket port 8085. Pastikan status koneksi WebSocket adalah *101 Switching Protocols* dan tidak terhambat CORS atau kegagalan otentikasi kunci Reverb.